

DEV:

Sommaire :

1- l'organisation du code

2- les responsabilités de chaque module

3- les interactions entre les composants

Pour ce projet, nous avons travaillé en collaboration avec Git et GitHub. , voici l'arborescence du projet (prise de GitHub) :

- Architecture du projet :

```
- .
-  └─ src
-      └─ game
-          ├── ASCII
-          │   ├── PrintBackPack.java
-          │   ├── PrintDungeon.java
-          │   ├── PrintEnemyRoom.java
-          │   ├── PrintFloor.java
-          │   ├── PrintHealer.java
-          │   ├── PrintHelper.java
-          │   ├── PrintHero.java
-          │   ├── PrintMerchant.java
-          │   └─ PrintTreasure.java
-          ├── controller
-          │   ├── GameLoop.java
-          │   ├── InteractionController.java
-          │   └─ MovmentController.java
-          ├── dungeon
-          │   ├── Coord.java
-          │   ├── Dungeon.java
-          │   ├── Floor.java
-          │   ├── Room.java
-          │   └─ rooms
-          │       ├── CorridorRoom.java
-          │       ├── EnemyRoom.java
-          │       ├── ExitRoom.java
-          │       ├── HealerRoom.java
-          │       ├── MerchantRoom.java
-          │       └─ TreasureRoom.java
-          └─ RoomType.java
```

```

-      |└─ enemies
-      |  |└─ Action.java
-      |  |└─ Enemy.java
-      |  |└─ EnemyType.java
-      |  |└─ RatWolf.java
-      |  |└─ Slime.java
-      |  └─ SmallRatWolf.java
-      |└─ hero
-      |  └─ Hero.java
-      |└─ interaction
-      |  |└─ Combat.java
-      |  |└─ CombatResult.java
-      |  |└─ EnemyController.java
-      |  |└─ ItemManager.java
-      |  └─ UICombat.java
-      |└─ items
-      |  |└─ armor
-      |  |  |└─ Armors.java
-      |  |  |└─ KnightShield.java
-      |  |  └─ RoughBuckler.java
-      |  |└─ consumables
-      |  |  |└─ Consumables.java
-      |  |  |└─ HealthStone.java
-      |  |  └─ TunaBox.java
-      |  |└─ Item.java
-      |  |└─ magic
-      |  |  └─ MagicWand.java
-      |  |└─ MagicBackPack.java
-      |  └─ weapons
-      |    |└─ Dart.java
-      |    |└─ JackCleaver.java
-      |    |└─ Weapon.java
-      |    └─ WoodenSword.java
-      └─ Main.java

```

Le donjon contient une liste d'étage , chaque étage est un objet Floor qui est une grille de 5*11 de Room .

Pour Room, nous avons créé une interface pour centraliser les différentes méthodes d'une pièce. De plus pour faciliter le travail, un RoomType (enum) a été créé.

Le hero est dans sa classe Hero où il y a toutes les données tel que HP, mana ,énergie , arme équipée. De plus les méthodes attack(), block(), heal() y sont aussi.

Pour la suite peut être on va créer une Classe dans le package hero pour gérer les actions qu'il fait .

Les Items , il y a 3 interfaces , Weapon , Armor, Consumables pour les différents types d'objets puis chacun de ces objets implémenteront aussi l'interface Item .

Le Sac à dos : est implémenté dans la Class MagicBackPack, et chaque objet du sac à dos est un item , on a donc différentes méthodes pour se remplir : `fitsInBackpack()` vérifie collision , `placeInBackpack()` , `add()`. Pour reproduire le plus fidèlement possible le jeu original , un item aura une abréviation tel que WS pour Wooden Sword , celle-ci prend 3*1 place dans le sac donc les 3 cases horizontales seront libellées WS .

Le combat : dans la classe Combat il y a la boucle principale du combat , UICOMBAT affichera le choix de l'arme ou de l'objet de défense du joueur , ItemManager gère les items utilisés par le héros , EnemyController exécute l'action de l'ennemi et enfin un enum CombatResult qui stocke WIN/LOSE.

Pourquoi séparer ?

- Facilite les tests et debug
- Permet un futur passage en interface graphique (Zen)
- Rend le moteur indépendant du reste du jeu

GameController: la class GmeLoop on y retrouvera la boucle principale du jeu , MovementController pour la gestion des déplacements

Le package ASCII : dedans il y a une classe pour chaque type de Room, pour le combat , le floor et le donjon qui fera l'affichage en ASCII .

On a choisi de suivre une architecture modulaire pour faciliter le plus possible la transition de ASCII -> ZEN , de plus tout séparer en package facilite grandement le débogage .

Cependant la version actuelle qui est fournie dans le zip , n'est pas la version finale de la phase 1 . On est retourné en arrière de 2 commits pour vous fournir une version compilable et jouable (ASCII) .

D'ici notre soutenance qui se tiendra jeudi nous espérons vous fournir une version qui correspond plus aux demandes de la phase 1 .